

Bericht

Jiaqi Wang

June 16, 2026

1 Einführung

In dieser Woche wurde die Ansteuerung eines Schrittmotors mit einem ESP32-S3 umgesetzt. Dabei wurden sowohl eine Open-Loop-Steuerung als auch eine Closed-Loop-Steuerung mithilfe eines Drehgebers realisiert. Zusätzlich wurde eine TCP-Kommunikation über WLAN implementiert, um den Motor drahtlos steuern zu können.

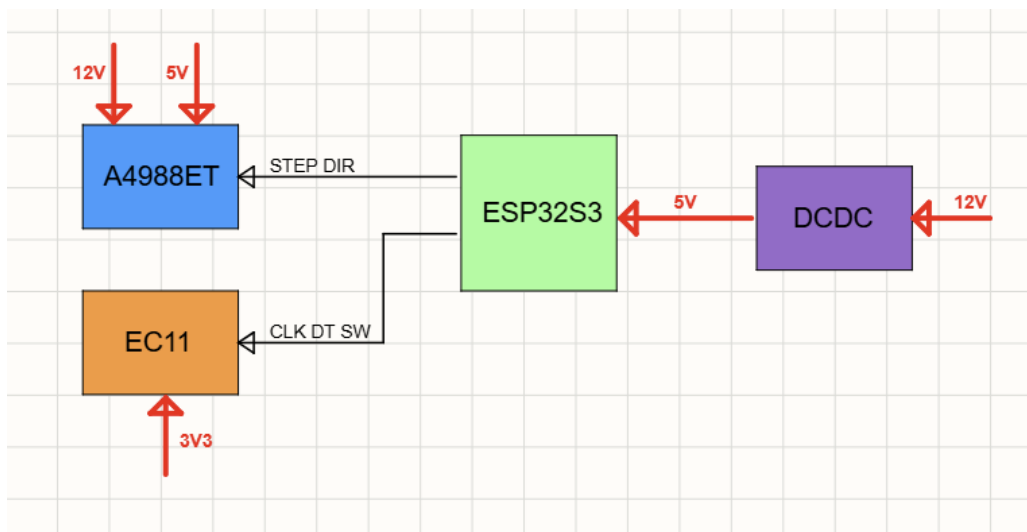


Figure 1: Systemübersicht des Versuchsaufbaus

2 Verwendete Komponenten

Table 1: Verwendete Hardware-Komponenten

Komponente	Typ / Modell	Funktion
Mikrocontroller	ESP32-S3 Dev Module	Hauptsteuerung
Schrittmotor	17HM5417	Rotationsantrieb
Motortreiber	A4988	Ansteuerung des Schrittmotors
Drehgeber	EC11 Encoder	Positionsrückmeldung
Netzgerät	12 V DC-Netzteil	Versorgung des Motors
DC/DC-Wandler	12 V / 5 V	Spannungswandlung für ESP32-S3

Die Spannungsversorgung des Systems erfolgt über ein externes 12 V-Netzgerät. Der Schrittmotor wird über den A4988-Treiber direkt mit 12 V betrieben.

Da der ESP32-S3 eine niedrigere Versorgungsspannung benötigt, wird zusätzlich ein DC/DC-Wandler verwendet, der die Eingangsspannung von 12 V auf 5 V reduziert. Dadurch können sowohl die Steuerungselektronik als auch die Motoransteuerung aus derselben Spannungsquelle versorgt werden.

3 Schrittmotor und Motortreiber

Für den Versuchsaufbau wurde ein Schrittmotor des Typs **17HM5417** mit einem Schrittwinkel von **1,8°** verwendet. Die Ansteuerung erfolgte über den Motortreiber **A4988**.

Im Vollschrittbetrieb beträgt die theoretische Schrittzahl für eine vollständige Umdrehung:

$$\frac{360^\circ}{1.8^\circ} = 200$$

Somit sollte der Motor nach 200 Schritten genau eine Umdrehung ausführen.

Die Ansteuerung des Schrittmotors erfolgt über die STEP- und DIR-Signale des A4988-Treibers. Das folgende Programmfragment zeigt die grundlegende Motorsteuerung.

```
void stepMotor(int steps, bool dir) {
    digitalWrite(DIR_PIN, dir);

    for (int i = 0; i < steps; i++) {
        digitalWrite(STEP_PIN, HIGH);
        delayMicroseconds(1000);
        digitalWrite(STEP_PIN, LOW);
        delayMicroseconds(1000);
    }
}
```

Während der ersten Tests wurde eine Open-Loop-Steuerung implementiert. Obwohl theoretisch 200 Schritte einer vollständigen Umdrehung entsprechen, rotierte der Motor nur

etwa 10° bis 20° . Eine mögliche Ursache ist die hohe mechanische Reibung der Lagerung. Daher wurde anschließend eine Closed-Loop-Steuerung mit Drehgeber entwickelt.

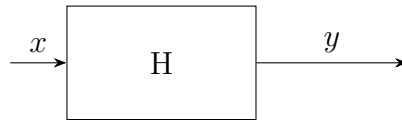


Figure 2: Blockdiagramm einer Open-Loop-Steuerung

Dabei beschreibt (x) den Sollwinkel und (y) die resultierende Position des Motors. Das Übertragungsglied (H) umfasst den ESP32-S3, den Motortreiber A4988 sowie den Schrittmotor.

Zur Verbesserung der Positioniergenauigkeit wurde ein EC11-Drehgeber in das System integriert. Der Drehgeber ist mechanisch mit der Motorwelle gekoppelt und liefert kontinuierlich Positionsinformationen an den ESP32-S3.

```

void IRAM_ATTR encoderISR() {
    int clkValue = digitalRead(ENCODE_CLK);
    int dtValue = digitalRead(ENCODE_DT);

    if (lastCLK != clkValue) {
        lastCLK = clkValue;

        if (clkValue != dtValue) {
            encoderCount++;
        } else {
            encoderCount--;
        }
    }
}

```

Die Signale CLK und DT des Encoders werden über Interrupts ausgewertet. Aus den erfassten Impulsen berechnet der ESP32 die aktuelle Winkelposition der Motorwelle. Anschließend wird diese Position mit dem vorgegebenen Sollwinkel verglichen.

Bei Abweichungen zwischen Soll- und Istwert erzeugt der ESP32 zusätzliche STEP-Signale für den A4988-Treiber, bis die gewünschte Position erreicht ist. Dadurch entsteht ein Closed-Loop-Regelkreis, der Positionsfehler automatisch korrigieren kann.

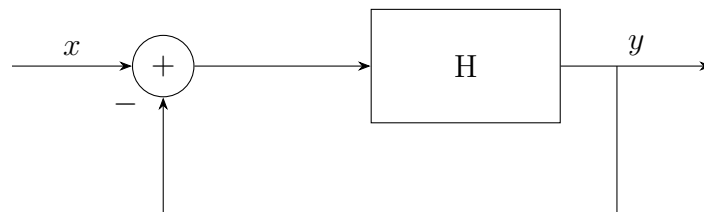


Figure 3: Blockdiagramm einer Closed-Loop-Steuerung

Im Gegensatz zur Open-Loop-Steuerung ermöglicht dieses Verfahren eine höhere Genauigkeit. Die aktuelle Positionsabweichung zwischen Sollwinkel und Istwinkel beträgt jedoch noch etwa 10° .

4 TCP-Kommunikation über WLAN

Neben der Motorsteuerung wurde eine TCP-basierte Kommunikation über WLAN implementiert. Der ESP32-S3 fungiert dabei als TCP-Server und wartet auf eingehende Verbindungen eines Clients.

Nach dem Verbindungsaufbau können Steuerbefehle über das Netzwerk an den ESP32-S3 gesendet werden. Die empfangenen Daten werden verarbeitet und anschließend zur Ansteuerung verschiedener Funktionen des Systems verwendet.

Zusätzlich wurde mit dem Qt-Framework eine einfache grafische Benutzeroberfläche entwickelt. Die Anwendung dient als TCP-Client und ermöglicht die drahtlose Kommunikation mit dem ESP32-S3.

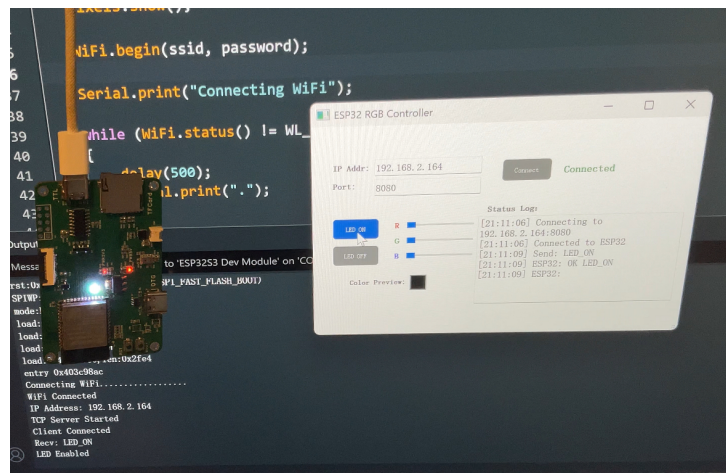


Figure 4: Qt-basierte Benutzeroberfläche zur Kommunikation mit dem ESP32-S3

Über die Benutzeroberfläche können Befehle an den Mikrocontroller gesendet werden. Im Rahmen eines ersten Tests wurde die auf dem ESP32-S3 integrierte WS2812-RGB-LED erfolgreich drahtlos angesteuert. Dadurch konnte die TCP-Kommunikation zwischen PC und Mikrocontroller erfolgreich verifiziert werden.